

Hybrid Quantum-Classical Cyber-Physical System for Hydroponic Lettuce Optimization

Overview

This repository contains the methodology and codebase for a hybrid quantum-classical optimization framework applied to a controlled-environment hydroponics system cultivating *Lactuca sativa* (Lettuce). The system bridges classical Deep Learning with Quantum Computing to form a continuous closed feedback loop.

Classical neural networks handle **perception** by parsing noisy time-series sensor data to predict a continuous Plant Health Index ($\mathcal{H}$). Utilizing this index, a Quantum Computer handles **prescriptive action** by mapping hardware configurations to a quantum optimization landscape, finding the exact operational state that maximizes lettuce health while strictly minimizing electricity consumption.

Core Architecture: The Hybrid Loop

- Classical AI (Perception):** A predictive time-series neural network maps environmental inputs to a decimal Health Index between 0.0 (critical) and 1.0 (optimal peak health).
- Quantum Optimizer (Action):** The Health Index informs a multi-objective cost function within a Quadratic Unconstrained Binary Optimization (QUBO) formulation.
- Variational Quantum Eigensolver (VQE):** A parameterized quantum circuit (Ansatz) interacts with a classical optimizer to iteratively search the mathematical landscape, safely minimizing kilowatt-hours (kWh) without dropping the predicted Health Index below a target threshold.

Dataset & Feature Architecture

The framework processes an integrated dataset that bridges hardware energy states with physical biological metrics. To develop and validate the pipeline before live deployment, the repository includes an automation script (`1_generate_data.py`) that synthesizes a high-fidelity biological digital twin, cleans sensor dropouts, and normalizes values to ensure algorithmic stability.

The data features are categorized into three operational layers:

1. Control Variables (Hardware/Energy Metrics)

- timestamp:** Date and time logging, critical for capturing temporal dependencies in the time-series model.
- pump_power_w:** Electrical wattage drawn by the vertical tower water pumps.
- led_power_w:** Electrical wattage drawn by the variable-spectrum LED arrays.
- cooler_power_w:** Electrical wattage drawn by the climate cooling system.

2. Environmental Variables (Sensor Feeds)

- water_ph:** Acidity level of the liquid nutrient solution.
- water_ec:** Electrical Conductivity, serving as a direct proxy for dissolved nutrient concentration.

- **ambient_temp:** Ambient air temperature within the greenhouse/growth chamber (measured in Celsius).
- **water_temp:** Core temperature of the reservoir nutrient solution (measured in Celsius).
- **humidity:** Percentage of ambient air moisture.

3. Target Variables (Lettuce Plant Health Indicators)

- **maceration_chlorophyll:** Quantified chlorophyll pigment concentration extracted via laboratory maceration procedures (measured in mg/g).
- **leaf_area_cm2:** Total surface area of the canopy layer, tracked via top-down computer vision profiling or manual sampling.
- **health_index (H):** The master dependent variable. It is a calculated continuous decimal ranging from 0.0 (lethal stress) to 1.0 (optimal physiological health) derived directly from the scaled interaction of chlorophyll levels and canopy leaf area expansion.

Mathematical Formulation

To map the physical hydroponics environment to a quantum processor, we translate the optimization problem into an **Ising Hamiltonian**. We define a **3-qubit system** representing our core hardware variables:

- **q_0 (Water Pump State, Z_0):** Maps to +1 (OFF) or -1 (ON).
- **q_1 (LED Array State, Z_1):** Maps to +1 (LOW) or -1 (HIGH).
- **q_2 (Cooler State, Z_2):** Maps to +1 (OFF) or -1 (ON).

1. The Classical Loss Function (Mean Squared Error)

For the classical neural network training, since the model predicts a continuous decimal Plant Health Index rather than a binary state, we use Mean Squared Error:

$$MSE = \frac{1}{N} \sum_{i=1}^N (\mathcal{H}_i - \hat{\mathcal{H}}_i)^2$$

2. The Multi-Objective Cost Function

The goal is to minimize a multi-objective cost function that balances electricity consumption (E) against a penalty (P) for endangering the predicted Lettuce Health Index (H):

$$C = \alpha \cdot E(Z_0, Z_1, Z_2) + \beta \cdot P(Z_0, Z_1, Z_2 \mid \mathcal{H})$$

(Where α and β are user-defined priority weights)

3. The Problem Hamiltonian

When mapped to Pauli-Z operators, the cost function yields the Problem Hamiltonian (H):

$$H = c_0 Z_0 + c_1 Z_1 + c_2 Z_2 + c_{01}(Z_0 \otimes Z_1) + c_{12}(Z_1 \otimes Z_2) + c_{02}(Z_0 \otimes Z_2)$$

- **Linear Terms ($c_i Z_i$):** Represent the base electricity consumption cost of running individual hardware components.

- **Quadratic Terms ($\sum_i c_i Z_i \otimes Z_j$):** Represent correlated penalties (e.g., dynamically elevating the "energy" of the system if the algorithm attempts to turn off both the pump and cooler while ambient temperature is dangerously high).

Quantum Optimization

The framework relies on the **Rayleigh-Ritz variational principle**, which states that the expectation value of the Hamiltonian for any parameterized trial state $|\psi(\theta)\rangle$ is always greater than or equal to the true ground state energy (lowest eigenvalue, E_0):

$$\langle \psi(\theta) | H | \psi(\theta) \rangle \geq E_0$$

The VQE algorithm iteratively updates the parameter vector θ to drive $\langle H \rangle_{\psi(\theta)}$ down to its minimum value. This mathematical ground state maps directly to the safest, most energy-efficient hardware configuration.

Algorithmic Pipeline & Tech Stack

To ensure computational efficiency and address the constraints of the Noisy Intermediate-Scale Quantum (NISQ) era, the following stack is utilized:

Component	Selected Technology	Academic Justification & Use
Classical Perception	Gated Recurrent Unit (GRU)	Detects rapid shifts in time-series environmental sensor data with significantly lower computational overhead than transformer models.
Quantum Algorithm	Variational Quantum Eigensolver (VQE)	The industry standard for near-term hybrid optimization. Highly publishable and tailored perfectly for current NISQ-era constraints.
Circuit Ansatz	RealAmplitudes (Linear Entanglement)	A parameterized hardware-efficient quantum circuit that minimizes gate depth, dramatically lowering the risk of decoherence errors on cloud-based quantum processors.
Classical Optimizer	SPSA	Simultaneous Perturbation Stochastic Approximation. Efficiently handles and navigates the inherent computational readout noise of cloud-based IBM quantum backends.
Benchmark Baseline	Simulated Annealing (SA)	Provides a direct, peer-reviewed classical point of comparison to demonstrate the

Component	Selected Technology	Academic Justification & Use
		algorithmic accuracy and convergence rate of the quantum framework.

Classical vs. Quantum Benchmark

To rigorously evaluate the framework's performance, a benchmarking module pits the Qiskit VQE against classical heuristic optimization algorithms (Simulated Annealing and Genetic Algorithms). The algorithms are compared side-by-side on convergence runtime, computational resource overhead, and the absolute accuracy of finding the global minimum. This establishes a baseline proving the viability of VQE as quantum hardware scales toward supremacy.

Repository Structure

- hydro_data.csv: Cleaned, normalized time-series dataset featuring control parameters, environmental sensors, and calculated lettuce health indices.
- 1_generate_data.py: Simulates the digital twin environment to synthetically construct the dataset, automate missing-value data cleaning, and handle scaling/normalization profiles.
- 2_train_model.py: Compiles the PyTorch GRU model to forecast the continuous Health Index.
- 3_run_quantum.py: Executes the Qiskit VQE optimizer loop on IBM Quantum infrastructure using SPSA.
- 4_benchmark.py: Runs Simulated Annealing and Genetic Algorithms to benchmark against the VQE performance.